Project Report

Winter Semester 2025-26

Computer Vision (CSE 3010)

SLOT – **D11+D12**

Project Type: Bring Your Own Project (BYOP)

Object Detection Project

Submitted by:

Name: **Aarya Butolia**

Reg. No: **23BAI10414**

Submitted to:

Faculty: Dr. Rajneesh Patel

1. Introduction

Object detection is one of the most important tasks in **Computer Vision**, a field of Artificial Intelligence that enables machines to interpret and understand visual information from the real world. It combines **image classification** and **localization**, meaning it not only identifies what objects are present in an image but also determines **where those objects are located**.

Traditional computer vision techniques relied on handcrafted features such as edges, textures, and shapes. However, these methods were slow and less accurate when dealing with complex scenes. With the advancement of **Deep Learning**, modern object detection systems use **Convolutional Neural Networks (CNNs)** to automatically learn features from images.

This project focuses on implementing a **real-time object detection system using** YOLOv8. The term **YOLO (You Only Look Once)** represents a single-stage detection method that processes the entire image at once, making it extremely fast compared to earlier multi-stage detectors.

The developed system supports:

- Static image detection
- Video-based detection
- Real-time webcam detection

This project demonstrates how modern deep learning models can be used to build efficient and scalable real-time detection pipelines suitable for real-world applications such as:

- Smart surveillance
- Automated monitoring
- Traffic analysis

- Industrial inspection

The main focus is to design a system that is **fast, accurate, flexible, and user-friendly**, making it suitable for deployment in practical environments.

---

2. Problem Statement

Manual object detection from images or videos is inefficient, time-consuming, and prone to human error. In many real-world scenarios such as surveillance, traffic monitoring, and industrial automation, thousands of images or video frames need to be processed continuously. Manual monitoring is not feasible in such situations.

Additionally, with the rapid increase in digital content generation, there is a strong need for **automated intelligent systems** capable of identifying objects in real time.

The key challenges addressed in this project include:

1. Real-Time Detection

Many traditional object detection systems are slow and cannot process live video streams efficiently. This project aims to build a system capable of processing frames quickly while maintaining high detection accuracy.

2. Multi-Input Processing

Most existing systems handle only a single input format. This project supports:

- Image files (.jpg, .png)
- Video files (.mp4, .avi)
- Live webcam feeds

This makes the system more flexible and widely applicable.

3. Visualization of Detection Results

For better interpretation of results, the system must display:

- Bounding boxes
- Class labels
- Confidence scores

This visual output helps users understand how well the system is performing.

4. Analytical Insights

Beyond detection, the system should provide statistical insights such as:

- Object frequency distribution
- Confidence score analysis
- Graphical representation of results

---

3. Objectives

The primary goal of this project is to design and implement a **real-time object detection system** using deep learning techniques.

The specific objectives include:

1. Implement YOLOv8-Based Detection

Develop a detection system using a pre-trained YOLOv8 model capable of identifying multiple objects within a single frame.

2. Support Multiple Input Sources

Enable detection from:

- Static images
- Pre-recorded videos

- Live webcam streams

This ensures versatility in different real-world scenarios.

3. Visualize Detection Outputs

Display results using:

- Rectangular bounding boxes
- Class labels
- Confidence scores

This helps verify the correctness of detections.

4. Analyze Detection Results

Generate graphical outputs such as:

- Bar charts showing object counts
- Histograms representing confidence scores

5. Develop an Efficient and User-Friendly System

Ensure the system is:

- Easy to use
- Modular in design
- Efficient in performance

---

4. Methodology

The methodology describes how the detection system works internally.

The core algorithm used is **YOLOv8**, which belongs to the family of **single-stage object detectors**.

Working Principle

1. Input Processing

The system accepts input from:

- Image files
- Video files
- Webcam streams

Images are resized to a fixed dimension (for example **640×640 pixels**) before being passed into the neural network.

---

2. YOLOv8 Model

The system uses the **pre-trained model**:

- **yolov8n.pt (Nano Model)**
- Trained on the **COCO dataset**

The COCO Dataset contains:

- 80 object categories
- Millions of labeled images
- Real-world scene diversity

Examples of detectable objects:

- Person
- Car
- Dog
- Bicycle
- Laptop

---

3. Single-Shot Detection

Unlike traditional two-stage detectors, YOLOv8 processes the entire image **in one pass**.

Advantages:

- Faster processing
- Reduced computation time
- Real-time performance

---

4. Object Detection Output

The model predicts:

- Object class label
- Confidence score
- Bounding box coordinates

Example Output:

Person — 0.92
Car — 0.87

---

5. Filtering

Low-confidence detections are removed using a threshold value such as:

Confidence Threshold = **0.5**

This improves detection quality.

---

6. Visualization

Detected objects are displayed using:

- Colored bounding boxes
- Text labels
- Confidence scores

---

7. Data Analysis

Detection results are stored and analyzed to produce:

- Object count distribution
- Confidence score trends

---

5. Workflow

The workflow explains the step-by-step execution of the system.

Step 1: Install Required Libraries

Required libraries include:

- Ultralytics
- OpenCV
- NumPy
- Matplotlib

Command:

pip install ultralytics opencv-python matplotlib numpy

Step 2: Load YOLOv8 Model

The YOLOv8 model is loaded into memory.

Example:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
```

Step 3: Provide Input

Input can be:

- Image file
- Video file
- Webcam feed

Step 4: Process Input Data

Frames are resized and converted into required format.

Step 5: Run YOLOv8 Detection

The model performs detection on input data.

Step 6: Extract Object Details

Extract:

- Class name
- Confidence value
- Bounding box coordinates

Step 7: Draw Bounding Boxes

Boxes are drawn using OpenCV.

Step 8: Save Annotated Output

Detected outputs are stored in files.

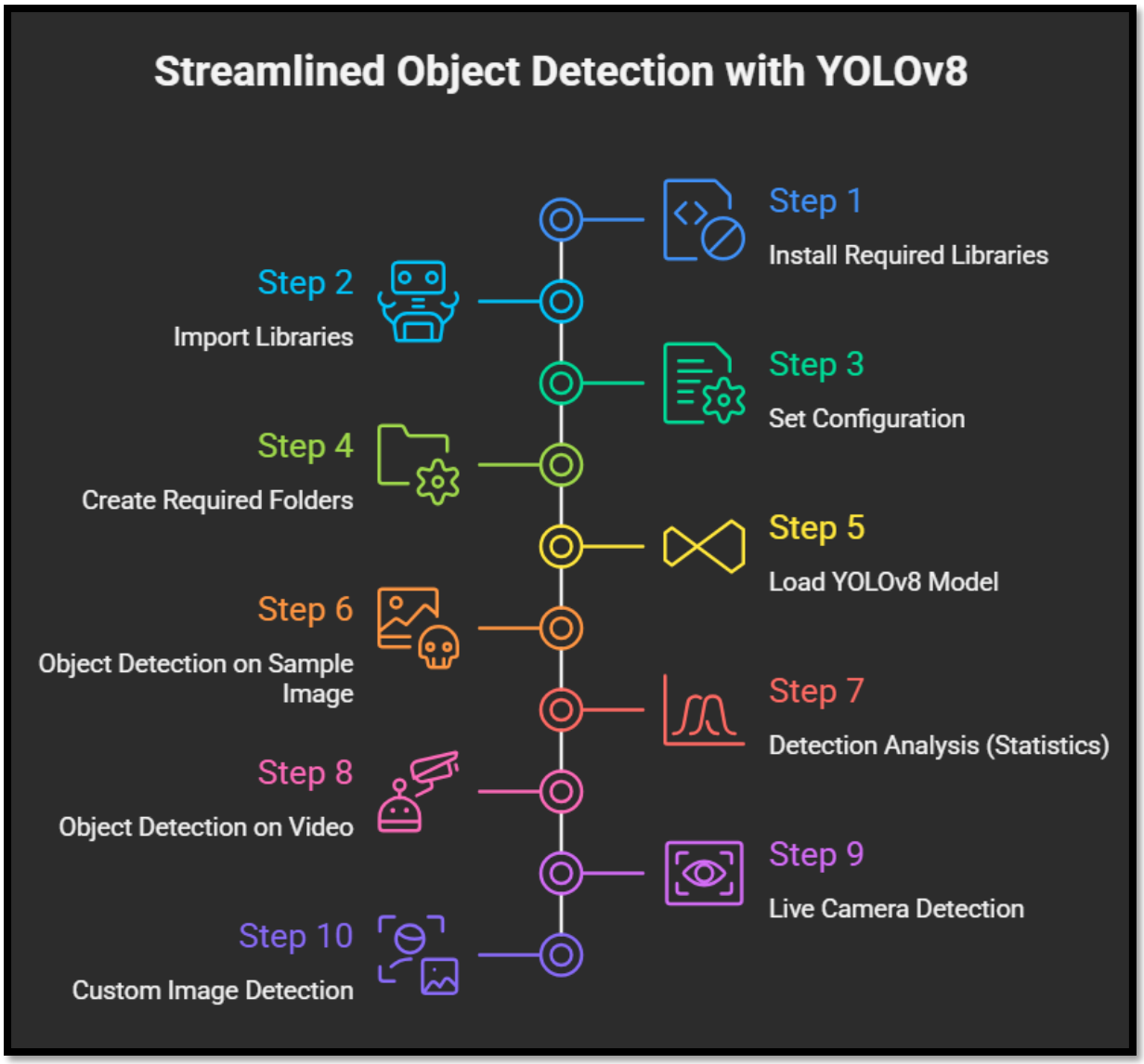
Step 9: Generate Graphical Analysis

Create:

- Bar charts
- Histograms

Step 10: Display Final Results

Outputs are displayed to the user.



---

6. System Design & Implementation

The system is implemented using **modular programming**, allowing each component to work independently.

Technologies Used (Expanded)

Python

Primary programming language used for:

- Model implementation

- Data processing
- Visualization

OpenCV

Used for:

- Image processing
- Video handling
- Drawing bounding boxes

Ultralytics YOLOv8

Provides:

- Pre-trained models
- Detection APIs

Matplotlib

Used for:

- Graph generation
- Visualization

NumPy

Used for:

- Matrix operations
- Numerical computations

Modules

1. Image Detection Module

Functions:

- Load image

- Perform detection
- Save annotated output

Example:

Input → image.jpg
Output → detected_image.jpg

---

2. Video Detection Module

Processes video frame-by-frame.

Steps:

- Read video frames
- Apply detection
- Combine frames into output video

---

3. Webcam Detection Module

Captures live feed from camera.

Features:

- Real-time detection
- Snapshot support

---

4. Analytics Module

Generates graphical outputs such as:

Bar Chart

Displays number of detected objects.

Example:

Person — 10
Car — 6

Histogram

Displays distribution of confidence scores.

---

7. Key Decisions Made

During the development of this object detection system, several important technical decisions were made to ensure that the system performs efficiently and remains suitable for real-time applications. These decisions helped improve performance, usability, and scalability.

1. Selection of YOLOv8 Model

The YOLOv8 model was selected as the core detection algorithm because it is one of the latest and most efficient real-time object detection models available. Compared to earlier models such as R-CNN and Faster R-CNN, YOLOv8 offers faster inference speed and improved accuracy. Since the objective of this project was to perform real-time detection, YOLOv8 was an appropriate choice.

2. Use of Pre-Trained Model Instead of Training from Scratch

Training deep learning models from scratch requires large datasets and powerful hardware such as GPUs. To reduce development time and computational requirements, a pre-trained YOLOv8 model trained on the COCO dataset was used. This decision allowed the system to detect common objects immediately without the need for additional training.

3. Selection of yolov8n (Nano Model)

Among different YOLOv8 variants (Nano, Small, Medium, Large), the yolov8n model was selected because it is lightweight and optimized for faster processing. This makes it suitable for systems that run on CPU, ensuring that the project can work even without high-end hardware.

4. Choice of Python Programming Language

Python was selected as the programming language because of its simplicity, readability, and strong support for machine learning and computer vision libraries. Python also provides easy integration with OpenCV and deep learning frameworks.

5. Use of OpenCV for Visualization

OpenCV was used for image and video processing because it provides efficient functions for drawing bounding boxes, reading video streams, and handling real-time camera input. It also supports multiple file formats, making it suitable for this project.

6. Confidence Threshold Selection

A confidence threshold value of **0.5** was selected to filter low-confidence detections. This decision improved detection accuracy by removing uncertain predictions and ensuring that only reliable detections were displayed.

7. Modular System Design

The system was designed using modular components such as image detection, video detection, webcam detection, and analytics modules. This design decision makes the system easier to maintain, debug, and extend in the future.

---

8. Results and Discussion

The developed system was tested on various input formats including images, videos, and live webcam streams.

Performance Analysis

The system achieved:

- 20–30 Frames Per Second (FPS) on CPU
- Faster performance on GPU

Performance depends on:

- System hardware
- Input resolution
- Model size

Accuracy Analysis

The model showed high detection accuracy for:

- Large objects
- Clearly visible objects

Performance decreases when:

- Lighting is poor
- Objects are very small
- Objects overlap heavily

Outputs Generated

The system produces:

1. Annotated images

2. Processed videos
3. Statistical graphs

Limitations (Expanded)

1. Poor performance in low lighting
2. Difficulty detecting very small objects
3. Requires GPU for faster processing
4. Performance affected by motion blur

---

9. Advantages

- Real-time processing capability
- High detection accuracy
- Supports multiple input formats
- Easy to integrate into existing systems
- Lightweight models available
- Scalable architecture

---

10. Applications

Real-world applications include:

---

1. Smart Surveillance

Detect suspicious activities automatically.

Used in:

- Airports
- Railway stations
- Public places

---

2. Traffic Monitoring

Detect:

- Vehicles
- Pedestrians
- Traffic violations

---

3. Autonomous Vehicles

Used to detect:

- Road signs
- Vehicles
- Obstacles

---

4. Industrial Automation

Detect:

- Defective products
- Missing components

---

5. Security Systems

Used for:

- Face detection

- Intrusion detection

---

11. Conclusion

This project successfully demonstrates the implementation of a **real-time object detection system** using YOLOv8. The system achieves efficient performance by processing entire images in a single pass using deep learning techniques.

Compared to traditional detection methods, YOLOv8 provides:

- Faster processing speed
- Higher detection accuracy
- Better scalability

The modular design ensures easy expansion and integration with future technologies. Overall, this project highlights the effectiveness of modern deep learning models in solving complex visual recognition problems.

---

12. Challenges Faced

During the implementation of this project, several practical challenges were encountered. These challenges provided valuable learning experiences and helped improve problem-solving skills.

1. Hardware Limitations

One of the main challenges was running real-time object detection on systems without GPU support. Deep learning models require significant computational power, and running the model on CPU resulted in slower processing speed. To address this issue, the lightweight yolov8n model was selected.

2. Installation and Dependency Issues

Installing required libraries such as Ultralytics, OpenCV, and Matplotlib required proper environment configuration. Some compatibility issues occurred during installation, which required troubleshooting and updating dependencies.

3. Real-Time Video Processing

Processing video frames continuously in real time required careful optimization. High-resolution videos increased processing time, making it necessary to resize frames to maintain acceptable speed.

4. Handling Multiple Input Formats

Supporting multiple input types such as images, videos, and webcam feeds required additional debugging and testing. Each input type required slightly different processing steps.

5. Visualization Synchronization

Drawing bounding boxes and labels on moving video frames without delay was challenging. Synchronizing frame processing with detection output required proper timing control.

6. Handling Low-Quality Inputs

Detection performance decreased when images had poor lighting, motion blur, or overlapping objects. Adjusting confidence thresholds helped reduce incorrect detections.

---

13. Future Scope

Possible future improvements include:

1. Custom Model Training

Train YOLOv8 on custom datasets.

Example:

- Medical images

- Industrial defects

---

2. Cloud Integration

Deploy system on cloud platforms for:

- Remote monitoring
- Data storage

---

3. Mobile Deployment

Implement detection on:

- Android devices
- Edge devices

---

4. Alert Systems

Trigger alerts when:

- Specific objects are detected
- Threshold values are exceeded

---

14. What I Learned

This project provided valuable hands-on experience in computer vision and deep learning. Several important technical and practical skills were developed during the development of this system.

1. Understanding Object Detection Concepts

Through this project, I learned how object detection combines classification and localization techniques. I understood how bounding boxes are used to locate objects within images.

2. Practical Implementation of YOLOv8

I gained practical knowledge of using YOLOv8 for detecting objects in images, videos, and real-time webcam streams. I also learned how pre-trained models can be used effectively in real-world applications.

3. Image and Video Processing Using OpenCV

This project helped me understand how OpenCV is used for handling images and videos, drawing bounding boxes, reading video frames, and processing live webcam streams.

4. Model Integration and Workflow Design

I learned how to integrate deep learning models into an application pipeline. This includes loading models, processing input data, running detection, and visualizing results.

5. Performance Evaluation Techniques

I gained experience in evaluating system performance using parameters such as Frames Per Second (FPS), detection accuracy, and confidence scores.

6. Debugging and Problem-Solving Skills

Several issues occurred during implementation, such as library installation errors and slow processing speeds. Solving these issues improved my debugging and troubleshooting skills.

7. Importance of System Optimization

I learned how selecting lightweight models and resizing input images can significantly improve system speed and performance.

---

15. Project Reflection

The development of this real-time object detection system was an enriching and practical learning experience. This project allowed me

to apply theoretical knowledge from computer vision and deep learning into a real-world implementation.

At the beginning of the project, understanding how object detection models work seemed complex. However, through continuous experimentation and practice, I gained clarity on how deep learning models process images and identify objects. Implementing YOLOv8 helped me understand how modern single-stage detectors achieve both speed and accuracy, making them suitable for real-time applications.

One of the most valuable aspects of this project was learning how to handle real-world data inputs such as images, videos, and live webcam streams. Working with multiple input formats improved my understanding of system design and the importance of modular programming. It also highlighted how different components of a system interact to produce final outputs.

Throughout the development process, I realized the importance of selecting appropriate tools and models based on system requirements. Choosing the lightweight yolov8n model helped maintain a balance between speed and accuracy, especially when running the system on CPU-based hardware. This experience strengthened my decision-making skills in selecting suitable technologies.

Another important lesson learned was the significance of debugging and testing. Several issues were encountered during installation, input handling, and visualization stages. Solving these problems improved my troubleshooting ability and increased my confidence in working with complex systems.

This project also emphasized the importance of performance optimization. Adjusting input resolution, filtering confidence values, and selecting efficient libraries helped improve system performance.

These optimizations demonstrated how small design decisions can significantly impact system efficiency.

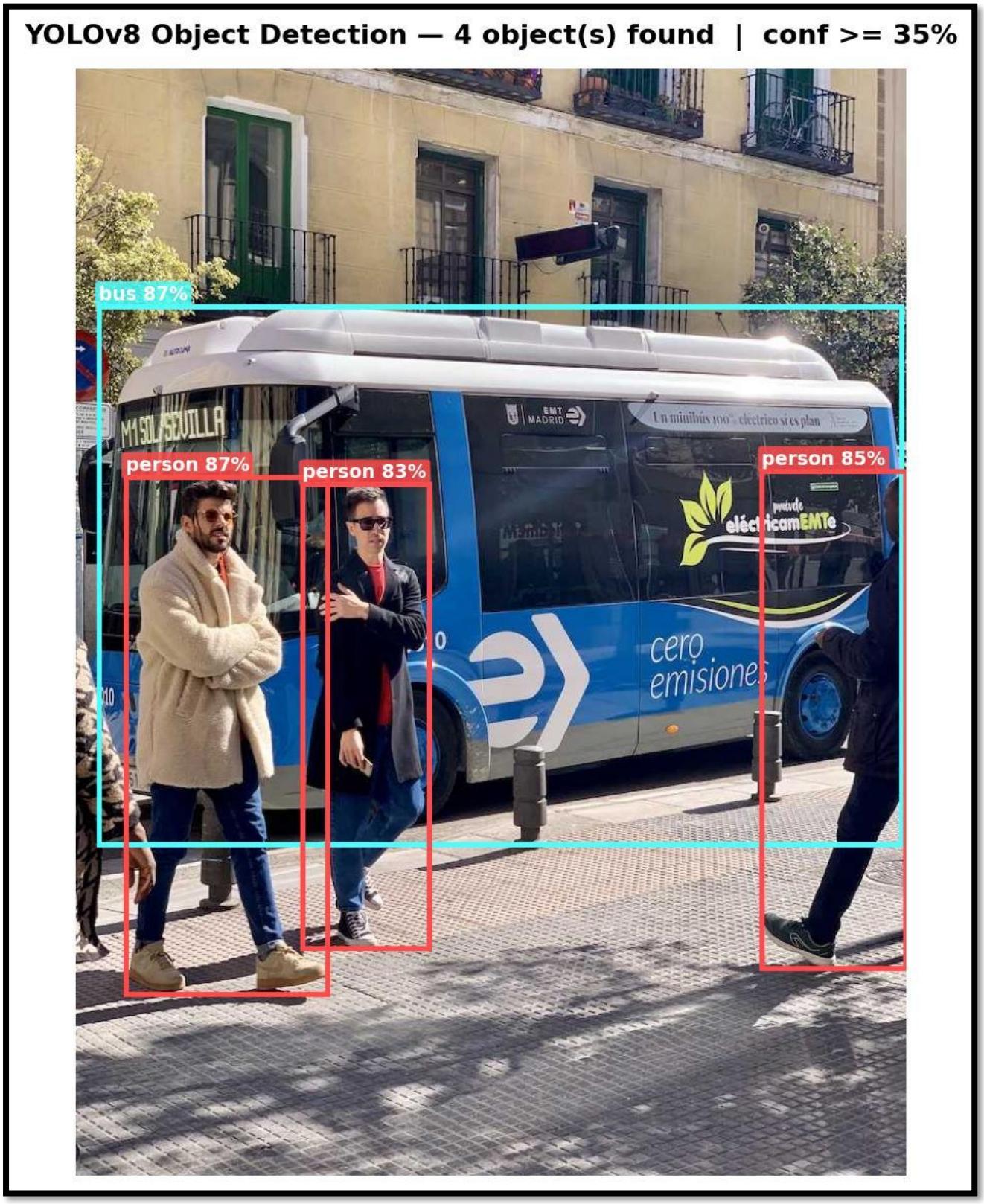
Overall, this project enhanced my understanding of computer vision concepts, deep learning workflows, and real-time system implementation. It also improved my programming, analytical thinking, and problem-solving skills. The knowledge gained through this project will be useful for future work in areas such as artificial intelligence, smart surveillance systems, and autonomous technologies.

---

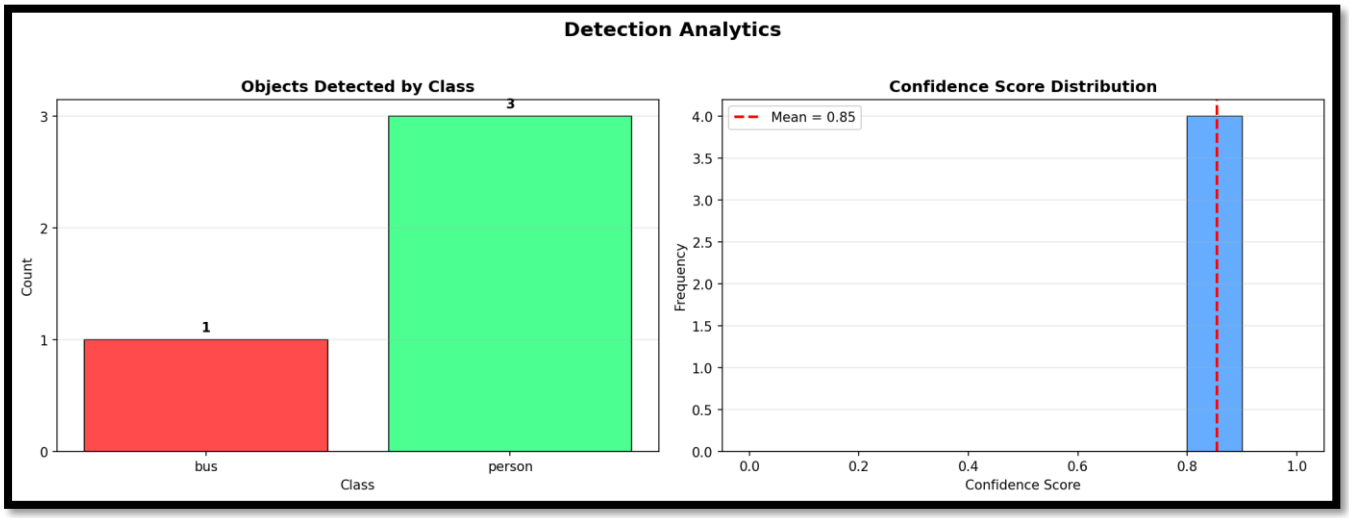
16. Output Samples

Webcam Detection Output

Image Detection Output



Detection Analysis



---

17. References

1. Ultralytics YOLOv8 Documentation
   https://docs.ultralytics.com/
2. COCO Dataset
   https://cocodataset.org/
3. OpenCV Documentation
   https://docs.opencv.org/
4. Python Documentation
   https://www.python.org/
5. Redmon, J., et al. (2016).
   "You Only Look Once: Unified, Real-Time Object Detection"
   IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
6. Bochkovskiy, A., et al. (2020).
   "YOLOv4: Optimal Speed and Accuracy of Object Detection"